PROJECT DOCUMENTATION

Credit Card Fraud Detection

Random Forest Based Fraud Monitoring Dashboard

Submitted by:

Santosh Kumar Shah
Surya Chand Yadav
Salim Shrestha

Submitted to:

Department of Computer Engineering
Kantipur City College, Putalisadak, Kathmandu
Affiliated to Purbanchal University

Supervisor:
Er. Sagar Dhakal

May, 2026

Declaration

We hereby declare that this project report entitled "**Credit Card Fraud Detection**" submitted to the Department of Computer Engineering is our original work and has not been submitted elsewhere for the award of any degree.

Santosh Kumar Shah
Surya Chand Yadav
Salim Shrestha

Acknowledgment

We would like to express our sincere gratitude to Er. Sagar Dhakal for his guidance and support throughout this project. We also thank the Department of Computer Engineering at Kantipur City College for providing an academic environment in which this work could be planned, implemented, tested, and documented.

Santosh Kumar Shah
Surya Chand Yadav
Salim Shrestha

Abstract

This project presents a credit card fraud detection system named Fraud Shield. The system combines a trained Random Forest classifier with a Flask-based monitoring dashboard for real-time and batch transaction analysis. Unlike a standalone machine learning script, the application includes operational features that are important in fraud monitoring: authentication, transaction preprocessing, behavior profiling, risk scoring, alert generation, verification workflow, dashboard metrics, CSV upload, and downloadable reports.

The model uses 15 engineered features derived from transaction time, amount, merchant category, channel, country, and customer behavior. Raw transaction records are transformed by a shared feature engineering module, scaled using a fitted StandardScaler, and classified by a RandomForestClassifier saved as `fraud_model.pkl`. The application computes a fraud probability, converts it into a model decision using a configurable threshold, combines it with behavior-based signals, and produces a risk score between 0 and 100. The system supports both single transaction checks through the dashboard/API and CSV batch detection through a manual upload workflow.

The training data stored in `creditcard_raw.csv` contains 60,994 transactions, including 890 fraud records, giving a fraud rate of approximately 1.46 percent. The model evaluation reported in the repository shows high overall accuracy while preserving strong fraud-class recall. This documentation is based on the implemented repository and the project research paper [1].

Contents

Declaration i

Acknowledgment ii

Abstract iii

1 Introduction 1

1.1 Overview . . . 1
1.2 Problem Statement . . . 1
1.3 Objectives . . . 2
1.4 Major Features . . . 2
1.5 Scope . . . 3
1.6 Limitations . . . 3
1.7 Organization of the Document . . . 3

2 Literature Review 4

2.1 Credit Card Fraud Detection . . . 4
2.2 Machine Learning for Fraud Detection . . . 4
2.3 Random Forest Model . . . 4
2.4 Behavior-Based Fraud Signals . . . 5
2.5 Research Basis . . . 5

3 System Analysis 6

3.1 Existing System . . . 6
3.2 Proposed System . . . 6
3.3 Functional Requirements . . . 6
3.4 Non-Functional Requirements . . . 7
3.5 Feasibility Study . . . 7
3.5.1 Technical Feasibility . . . 7
3.5.2 Economic Feasibility . . . 7
3.5.3 Operational Feasibility . . . 8

4 Methodology **9**
4.1 Dataset . . . 9
4.2 Input Fields . . . 9
4.3 Feature Engineering . . . 9
4.4 Model Training . . . 10
4.5 Prediction Pipeline . . . 11
4.6 Risk Score Calculation . . . 11

5 System Design **13**
5.1 Architecture . . . 13
5.2 Data Flow . . . 13
5.3 API Design . . . 14
5.4 Persistence Design . . . 14
5.5 User Interface Design . . . 15

6 Implementation **16**
6.1 Development Environment . . . 16
6.2 Authentication . . . 16
6.3 Transaction Validation . . . 16
6.4 Feature Encoding . . . 16
6.5 Behavior Analysis . . . 17
6.6 Prediction and Risk . . . 17
6.7 Alerts and Verification . . . 17
6.8 CSV Batch Detection . . . 17
6.9 Reporting . . . 18

7 Testing and Results **19**
7.1 Smoke Testing . . . 19
7.2 Model Evaluation . . . 19
7.3 Sample Input Results . . . 20
7.4 Discussion . . . 20

8 Conclusion and Future Work **21**
8.1 Conclusion . . . 21
8.2 Future Enhancements . . . 21

References **21**

List of Tables

4.1 Model Features 10
4.2 Random Forest Training Configuration 11

5.1 Main Repository Components 13
5.2 Application Routes 14

7.1 Confusion Matrix 19
7.2 Classification Report 20

Chapter 1

Introduction

1.1 Overview

Credit card fraud detection is a classification problem in which a system must identify whether a transaction is genuine or fraudulent. The difficulty of the task comes from class imbalance, changing fraud behavior, and the need for rapid decisions. The repository implements a practical fraud monitoring dashboard rather than only a notebook experiment. The main application is written in Flask and uses a trained Random Forest model to classify transactions.

The system is organized around the file `app.py`. It loads the model artifact `fraud_model.pkl` and the scaler artifact `scaler.pkl`, accepts transaction inputs, engineers the required features, predicts fraud probability, assigns a risk score, creates alerts, stores monitoring history, and exposes dashboard/reporting APIs. A shared module, `feature_engineering.py`, keeps training-time and inference-time feature definitions consistent.

1.2 Problem Statement

Manual inspection of card transactions is slow and unreliable when transaction volume is high. A fraud detection system must process transactions quickly, recognize risky behavior, reduce missed fraud cases, and still allow human review for high-risk or uncertain cases. The project addresses this problem by building an application that combines machine learning prediction with practical monitoring workflows.

The system must handle two types of input. First, an operator should be able to submit a single transaction through the dashboard and immediately see fraud probability, prediction label, risk score, behavior signals, and verification status. Second, an operator should be able to upload a CSV file containing multiple transactions and receive a processed summary with errors, alerts, and updated dashboard metrics.

1.3 Objectives

- Develop a Random Forest based classifier for credit card fraud detection.
- Engineer meaningful transaction and behavior features instead of relying only on raw input fields.
- Build a Flask dashboard for authenticated fraud monitoring.
- Provide single transaction prediction and CSV batch fraud detection.
- Generate risk scores, alerts, transaction history, and downloadable CSV reports.
- Maintain consistency between model training and serving through shared feature definitions.
- Provide smoke tests that verify the main application routes and prediction workflow.

1.4 Major Features

- **Authentication:** Operators must sign in before accessing prediction and dashboard APIs.
- **Machine learning prediction:** A Random Forest classifier returns fraud probability and a binary label.
- **Behavior analysis:** Customer profiles track previous amounts, countries, devices, merchant categories, and recent transaction counts.
- **Risk scoring:** Model output and behavior signals are combined into a 0 to 100 risk score.
- **Real-time alerts:** Fraud predictions and high-risk transactions create alert records.
- **Verification workflow:** Operators can approve, block, review, or mark transactions as false positives.
- **CSV batch upload:** Multiple transactions can be processed from a structured CSV file.
- **Reports:** The system can export monitored transactions as `fraud_report.csv`.

1.5 Scope

The scope of this project includes the training and serving of a Random Forest model, a web dashboard, authenticated APIs, persistence through a JSON store, and test coverage through the included smoke-test script. The implementation is suitable for academic demonstration and for explaining how a fraud monitoring workflow can be built around a machine learning model.

1.6 Limitations

- The repository uses `data_store.json` for persistence instead of a production database.
- Authentication uses simple username/password checking and should be strengthened before production use.
- The dataset in the repository is synthetic and may not capture all real banking fraud patterns.
- Model drift, audit logging, encryption, rate limiting, and role-based access control are outside the current implementation.
- The model threshold is configurable but currently uses a default fraud threshold of 0.5.

1.7 Organization of the Document

Chapter 2 presents the literature and conceptual background. Chapter 3 describes the requirements and feasibility. Chapter 4 explains the methodology, feature engineering, model training, and prediction workflow. Chapter 5 presents the system design. Chapter 6 describes implementation details from the repository. Chapter 7 summarizes testing and results. Chapter 8 concludes the project and lists future enhancements.

Chapter 2

Literature Review

2.1 Credit Card Fraud Detection

Credit card fraud detection is commonly modeled as a supervised binary classification task. Each transaction is represented by features such as amount, time, merchant behavior, channel, and geographical indicators. The target class indicates whether the transaction is fraudulent. A key challenge is class imbalance: fraud transactions usually represent only a small fraction of the full dataset, so a model can obtain high accuracy while still missing important fraud cases.

2.2 Machine Learning for Fraud Detection

Machine learning methods are useful because they can learn patterns from historical transaction data and generalize to new transactions. Common algorithms include Logistic Regression, Decision Trees, Random Forests, Support Vector Machines, Gradient Boosting models, and neural networks. For an academic fraud detector, Random Forest is a suitable choice because it handles nonlinear relationships, is robust to noisy features, and provides strong performance without the complexity of deep learning.

2.3 Random Forest Model

Random Forest is an ensemble learning method that builds many decision trees and combines their predictions. Each tree is trained on a bootstrapped sample of the data and considers a random subset of features at each split. This reduces overfitting compared with a single decision tree and improves generalization. In this repository, `train_model.py` trains a RandomForestClassifier with 200 trees, class-weight balancing, maximum depth 12, and a fixed random seed for reproducibility.

2.4 Behavior-Based Fraud Signals

Fraud detection improves when the model considers behavior beyond a single transaction. The implemented system tracks customer-level profiles, including previous transaction count, average amount, maximum amount, known devices, known countries, known merchant categories, and recent transaction velocity. This behavior analysis helps identify suspicious changes such as a new country, a new device, night-time transaction, high amount, or rapid repeated transactions.

2.5 Research Basis

The project research paper supports the use of machine learning, specifically the Random Forest model, for credit card fraud detection [1]. The repository extends this idea by turning the model into a working monitoring application with dashboard, alerts, verification, batch upload, and report generation.

Chapter 3

System Analysis

3.1 Existing System

Traditional fraud review relies on rule-based checks or manual investigation. These approaches can be useful, but they are limited when fraud behavior changes or transaction volume increases. Static rules may create many false positives or fail to identify new fraud patterns. Manual review also requires significant time and effort.

3.2 Proposed System

The proposed system is a web-based credit card fraud detector named Fraud Shield. It uses a Random Forest model for prediction and adds a monitoring layer around the model. Each transaction passes through validation, behavior analysis, feature engineering, scaling, model prediction, risk scoring, alert generation, profile update, and persistence. Operators interact with the system through a browser dashboard and API endpoints.

3.3 Functional Requirements

Requirement	**Description**
Login and logout	The system must authenticate operators before protected routes are accessed.
Single prediction	The system must accept transaction details and return fraud probability, label, risk score, and behavior signals.
CSV upload	The system must process CSV files containing multiple transactions.

Dashboard metrics	The system must show total transactions, fraud predictions, high-risk transactions, open alerts, verified transactions, and average risk.
Alert handling	The system must generate and acknowledge alerts for suspicious transactions.
Verification	The system must allow transactions to be approved, blocked, reviewed, or marked as false positives.
Report export	The system must provide a downloadable CSV report of monitored transactions.

3.4 Non-Functional Requirements

- **Usability:** The dashboard should be easy for an operator to use without command-line interaction.
- **Reliability:** Model and scaler artifacts must load successfully before prediction.
- **Maintainability:** Training and inference must share the same feature order and feature names.
- **Performance:** Predictions should be lightweight enough for real-time dashboard use.
- **Security:** Protected endpoints must reject unauthenticated requests.
- **Auditability:** Transaction records should include operator, verification status, behavior signals, and timestamps.

3.5 Feasibility Study

3.5.1 Technical Feasibility

The project is technically feasible because it uses well-supported Python libraries: Flask for the web application, Pandas and NumPy for data processing, Scikit-learn for machine learning, and Joblib for model persistence. The repository already contains trained artifacts and a smoke-test script, so the system can be run and tested locally.

3.5.2 Economic Feasibility

The tools used in this project are open source and do not require licensing fees. The system can run on a standard development machine, making it economically feasible for academic demonstration.

3.5.3 Operational Feasibility

The dashboard design supports operators by presenting prediction results, alerts, and verification actions in one place. CSV batch upload also makes the system practical for testing multiple transactions without writing API calls manually.

Chapter 4

Methodology

4.1 Dataset

The repository contains the dataset file `creditcard_raw.csv`. It has 60,994 transaction records covering dates from 2023-01-01 to 2023-12-31. The target column is `is_fraud`. The dataset contains 890 fraud transactions, which is approximately 1.46 percent of the data. This imbalance is realistic for fraud detection and is handled through model training choices such as class-weight balancing.

4.2 Input Fields

The application requires the following transaction fields for prediction:

- `transaction_date`
- `transaction_time`
- `amount`
- `merchant_category`
- `country`
- `channel`

Optional metadata fields include `customer_id`, `card_last4`, `merchant`, and `device_id`. These fields support behavior analysis, dashboard display, and report generation.

4.3 Feature Engineering

The file `feature_engineering.py` defines the model feature list and the function `engineer_single_transaction()`. It converts raw transaction data and behavior

signals into the exact 15 features expected by the trained model.

Table 4.1: Model Features

Feature	Meaning
`hour`	Hour extracted from transaction time.
`day_of_week`	Day index extracted from transaction date.
`is_weekend`	Indicates Saturday or Sunday.
`is_night`	Indicates transaction between 00:00 and 05:59.
`amount`	Transaction amount.
`amount_log`	Natural log transform of amount using `log1p`.
`merchant_category_enc`	Encoded merchant category.
`channel_enc`	Encoded transaction channel.
`country_enc`	Encoded country.
`txn_count_last_1h`	Previous customer transactions in the last hour.
`txn_count_last_24h`	Previous customer transactions in the last 24 hours.
`avg_amount_prev`	Customer average amount before the current transaction.
`amount_ratio`	Current amount divided by previous average amount.
`is_new_country`	Indicates a country not previously seen for the customer.
`is_new_category`	Indicates a merchant category not previously seen for the customer.

4.4 Model Training

Training is implemented in `train_model.py`. The script loads `creditcard_raw.csv`, validates required columns, computes the engineered features, scales them with StandardScaler, splits the data using a stratified 80/20 train-test split, and trains a RandomForestClassifier.

Table 4.2: Random Forest Training Configuration

Parameter	**Value**
Model	RandomForestClassifier
Number of trees	200
Class weight	balanced
Maximum depth	12
Random state	42
Parallel jobs	-1
Test size	20 percent
Cross-validation	5-fold StratifiedKFold recall

4.5 Prediction Pipeline

The application prediction pipeline is implemented by `monitor_transaction()` in `app.py`. The workflow is:

1. Validate and sanitize the incoming transaction.
2. Load the customer behavior profile before updating it.
3. Analyze behavior signals such as new device, new country, high amount, and transaction velocity.
4. Engineer the 15 model features.
5. Scale features using `scaler.pkl`.
6. Predict fraud probability using `fraud_model.pkl`.
7. Convert probability into a binary prediction using the configured threshold.
8. Calculate risk score and risk level.
9. Create an alert when the transaction is model-flagged or high risk.
10. Update the customer behavior profile.
11. Save transaction, alert, and profile state to `data_store.json`.

4.6 Risk Score Calculation

The risk score combines model probability, model decision, and behavior risk points. The implemented calculation is:

$$\text{risk score} = \min(100, \text{round}(72p + f + b))$$

where p is the model fraud probability, f is 10 when the model predicts fraud and 0 otherwise, and b is the behavior component capped at 28 points.

Risk levels are assigned as follows:

- Low: score below 35
- Medium: score from 35 to 64
- High: score from 65 to 84
- Critical: score 85 or above

Chapter 5

System Design

5.1 Architecture

The system follows a client-server architecture. The browser dashboard communicates with the Flask backend. The backend validates transactions, runs preprocessing, uses the machine learning artifacts, updates in-memory and JSON-based state, and returns JSON responses to the frontend.

Table 5.1: Main Repository Components

Component	Purpose
`app.py`	Flask server, routes, authentication, prediction pipeline, alerts, reports, and persistence.
`feature_engineering.py`	Shared feature definitions and single-transaction feature engineering.
`train_model.py`	Retraining script for model and scaler artifacts.
`fraud_model.pkl`	Trained Random Forest classifier.
`scaler.pkl`	Fitted StandardScaler for the 15 model features.
`templates/index.html`	Dashboard UI for login, prediction, CSV upload, alerts, and verification.
`smoke_test.py`	End-to-end checks for the Flask application.
`sample_upload.csv`	Example CSV file for batch detection.
`sample_transactions.json`	Example normal and fraud transaction payloads.
`data_store.json`	Runtime persistence for transactions, alerts, and behavior profiles.

5.2 Data Flow

The data flow begins when an operator submits a transaction through the dashboard or uploads a CSV file. For each transaction, the backend creates a sanitized transaction object and metadata object. It then analyzes customer behavior, engineers

the model features, applies scaling, predicts fraud probability, calculates risk, creates alerts if necessary, updates the behavior profile, and saves the resulting record.

5.3 API Design

Table 5.2: Application Routes

Method	Endpoint	Purpose
GET	`/`	Render login page or dashboard.
POST	`/login`	Authenticate operator.
POST	`/logout`	Clear session.
GET	`/health`	Return application status, threshold, model files, and feature list.
POST	`/predict`	Process one transaction and return fraud/risk result.
GET	`/api/dashboard`	Return dashboard metrics, recent transactions, alerts, and profiles.
GET	`/api/transactions`	Return monitored transactions.
GET	`/api/alerts`	Return active alerts.
POST	`/api/alerts/<alert_id>/acknowledge`	Mark an alert as acknowledged.
POST	`/api/transactions/<transaction_id>/verify`	Update transaction verification status.
GET	`/api/sample-csv`	Download sample batch upload CSV.
POST	`/api/upload-csv`	Process uploaded CSV transactions.
GET	`/api/report.csv`	Download monitored transaction report.

5.4 Persistence Design

The application stores runtime state in memory and writes it to `data_store.json`. The state includes monitored transactions, alerts, and behavior profiles. Profiles contain transaction counts, amount statistics, recent timestamps, known devices, known countries, and known merchant categories. This design is simple and adequate for a demonstration project, but a production system should use a database.

5.5 User Interface Design

The dashboard in `templates/index.html` contains the following major areas:

- Login panel for unauthenticated users.
- Transaction monitoring form for single prediction.
- Prediction result cards for label, fraud probability, risk score, and verification status.
- CSV upload panel for batch fraud detection.
- Behavior signals and preprocessing summary.
- Alert queue and dashboard metrics.
- Recent transaction table with verification actions.

Chapter 6

Implementation

6.1 Development Environment

The project is implemented in Python. The required dependencies are listed in `requirements.txt`: Flask, Pandas, NumPy, Scikit-learn, and Joblib. The application runs locally with:

```
python app.py
```

and serves the dashboard at `http://127.0.0.1:5000/`.

6.2 Authentication

The application uses session-based authentication. The default credentials are `admin` and `admin123`. These values can be changed through environment variables `APP_USERNAME` and `APP_PASSWORD`. Protected endpoints use the `require_auth` decorator, which returns HTTP 401 for unauthenticated requests.

6.3 Transaction Validation

The function `parse_transaction()` validates required fields and ensures that `amount` is a non-negative finite number. It also fills optional metadata with safe defaults when values are missing. This prevents the model pipeline from receiving incomplete or malformed transaction objects.

6.4 Feature Encoding

The feature engineering module contains fixed category lists for merchant categories, channels, and countries. Unknown values are mapped to the final `other` category.

The order of these lists is important because the encoded integer values are part of the model input.

6.5 Behavior Analysis

The function `analyze_behavior()` computes behavior points and signal descriptions. It checks whether the customer is new, whether the amount is much higher than previous average, whether the device, country, or merchant category is new, whether recent transaction count is high, whether the amount is large, and whether the transaction occurs at night.

6.6 Prediction and Risk

The `score_ml()` function calls `predict_proba()` on the Random Forest model and uses the configured threshold to decide whether the transaction is fraud. The `calculate_risk()` function combines model probability and behavior points to produce the final operational risk score. This means an unusual but model-benign transaction can still become high risk if behavior indicators are strong.

6.7 Alerts and Verification

The system creates an alert when the transaction is predicted as fraud or when the risk score is at least 65. High-risk transactions are placed in `PendingReview`. Operators can update verification status through the dashboard as Approved, Blocked, Reviewed, or False Positive. When a transaction is verified, related alerts are acknowledged.

6.8 CSV Batch Detection

The CSV upload endpoint reads an uploaded file with Pandas, enforces a maximum of 500 rows, checks required columns, and processes each row through the same monitoring pipeline used by single prediction. Each successful row returns transaction ID, customer ID, amount, label, fraud probability, risk score, risk level, and verification status.

6.9 Reporting

The report endpoint creates a CSV file with transaction ID, timestamp, customer ID, merchant, amount, fraud probability, model prediction, risk score, risk level, verification status, and behavior signals. This makes the dashboard output easy to preserve or analyze in spreadsheet software.

Chapter 7

Testing and Results

7.1 Smoke Testing

The repository includes `smoke_test.py`, which uses the Flask test client for end-to-end checks. The script verifies:

- `/health` returns status 200 and a valid feature count.
- Login succeeds with correct credentials and fails with wrong credentials.
- A normal sample transaction returns the expected Normal label.
- A fraud sample transaction returns the expected Fraud label.
- Fraud probability is within the range 0 to 1.
- Risk score is within the range 0 to 100.
- Behavior and preprocessing fields are present.
- Dashboard, sample CSV, CSV upload, report download, logout, and authentication checks work.

7.2 Model Evaluation

The model performance reported in the repository is based on a 20 percent stratified test split. The dataset contains 60,994 synthetic transactions with 890 fraud transactions. The reported confusion matrix is shown below.

Table 7.1: Confusion Matrix

	Predicted Normal	**Predicted Fraud**
Actual Normal	12005	16
Actual Fraud	12	166

Table 7.2: Classification Report

Class	**Precision**	**Recall**	**F1-score**	**Support**
Normal	0.999	0.999	0.999	12021
Fraud	0.912	0.933	0.922	178
Accuracy		0.998		12199
Macro average	0.956	0.966	0.961	12199
Weighted average	0.998	0.998	0.998	12199

The repository reports a 5-fold cross-validation recall of 0.9242 ± 0.0271. These results indicate that the model detects most fraudulent transactions while keeping false positives low enough for dashboard-based verification.

7.3 Sample Input Results

The repository includes two sample JSON payloads. The normal sample uses a grocery transaction of 45.20 in the United States. The fraud sample uses a cash transaction of 500.00 from Russia at night on an unknown device. The smoke test expects the first sample to be classified as Normal and the second as Fraud.

7.4 Discussion

The combination of model probability and behavior analysis is important. The Random Forest provides the statistical prediction, while behavior scoring provides operational context. This design supports practical fraud review by exposing both the model output and the reasons a transaction may be risky.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

The project successfully implements a credit card fraud detection system using a Random Forest model and a Flask dashboard. The repository demonstrates an end-to-end workflow: data preparation, feature engineering, model training, prediction serving, risk scoring, alerting, verification, CSV batch detection, and reporting. By using shared feature definitions, the project reduces the risk of mismatch between model training and application inference.

The system is suitable for academic demonstration because it is runnable, testable, and connected to an operator-facing interface. The model evaluation shows strong performance on the available dataset, and the dashboard provides the workflow needed to inspect, verify, and report suspicious transactions.

8.2 Future Enhancements

- Replace `data_store.json` with PostgreSQL or MySQL for reliable persistence.
- Add hashed passwords, role-based access control, and stronger session security.
- Add audit logs for all alert and verification actions.
- Add model versioning and store which model scored each transaction.
- Monitor model drift and retrain with real-world transaction data.
- Add explainability features such as feature importance or per-transaction explanations.
- Add API rate limiting and HTTPS deployment configuration.
- Expand testing with unit tests for feature engineering and risk scoring.

References

[1] Santosh Kumar Shah, Surya Chand Yadav, and Salim Shrestha. Credit card fraud detection using random forest model. Research paper, Kantipur City College, Putalisadak, Kathmandu, 2025. Project research paper on credit card fraud detection using a Random Forest classifier.